Seeing the Symphony

System Architecture Document

Raffaele Cipriano · Washburn University

Version 0.1 — March 2026

1. Project Overview

Seeing the Symphony is a software research project that generates print-quality visual artifacts from orchestral scores. The primary output is a large-format lobby installation: a printed visualization displayed at the concert hall before a performance, giving non-musician audience members a visual map of the symphony they are about to hear.

The system consists of two phases:

- Phase 1 — Authoring Tool: An interactive local web application where a conductor loads a score, reviews auto-generated annotations, tweaks visual parameters, selects a template, and exports a high-resolution print file.
- Phase 2 — Lobby Artifact: The exported print. A static, self-explanatory visual output displayed in the concert hall.

The project bridges orchestral conducting practice, computational music analysis, and data visualization, representing the conductor's analytical interpretation of a score as a visual object.

2. System Architecture

The system is organized into four distinct layers that communicate in a unidirectional pipeline, with an iterative loop in the authoring phase.

See: diagram_components.svg for the full component diagram.

See: diagram_sequence.svg for the data flow sequence diagram.

2.1 Input Layer

Input Layer — Three Sources

- MusicXML File: primary score data — pitch, time, duration, basic dynamics and articulation where encoded
- Interpretation File (JSON): conductor annotation layer — attack, release, playing technique, phrase intent
- Harmonic Analysis Layer: key areas, modulation windows, section boundaries — auto-generated, manually refinable

2.2 Analysis Pipeline

Implemented in Python using the music21 library. Runs server-side via a FastAPI local server. Produces a unified data model consumed by the authoring UI.

Module	Responsibility
Score Parser	Ingests MusicXML; extracts note objects with pitch, duration, dynamics, and articulation data
Harmonic Analyzer	Detects key areas, modulation windows, and mode; maps to color vocabulary
Phrase Segmenter	Identifies section boundaries and phrase arcs for background layer and arc markers
Auto-Annotator	Generates draft interpretation JSON from score data; conductor refines in UI
Data Model	Unified note/section/parameter objects consumed downstream
Collision Resolver	Pre-render pass: detects spatial overlaps, attaches collision strategy and blend metadata to note objects
FastAPI Server	Local REST API bridge between Python pipeline and React browser UI

2.3 Authoring UI

Implemented in React with Konva.js for canvas rendering. Runs in the browser. Communicates with the Python backend via REST API.

Authoring UI — Panels

- Score Import Panel: load MusicXML, trigger analysis pipeline, display status
- Annotation Editor: review and refine the auto-generated interpretation file
- Parameter Controls: lane height mode, vertical layout mode, color palette
- Template Selector: Piano Roll, Spiral, and future extensible templates
- Live Preview Canvas: real-time visual feedback as parameters are adjusted
- Export Controls: trigger high-resolution render, select output format

2.4 Output / Rendering Layer

High-resolution export is handled by a Python rendering engine using the Cairo graphics library, ensuring print-quality output independent of screen resolution. The authoring UI preview uses the browser canvas for speed; the final export uses Cairo for fidelity.

Output Type	Specification
PDF	Vector, print-ready, large format (up to poster size)
PNG	Raster, 300+ dpi minimum
SVG	Scalable vector, archival quality
Templates	Piano Roll (horizontal panoramic) · Spiral (radial) · extensible

3. Visual Parameter System

Visual parameters are organized in two layers: a foreground layer encoding note-level data, and a background layer encoding form and harmonic structure.

3.1 Foreground Layer — Note Level

Parameter	Visual Encoding
Pitch	Vertical position within lane (higher pitch = higher position)
Time	Horizontal position, left to right, proportional to clock time
Duration	Width / length of the note element
Dynamics	Brightness, opacity, or size of the note element
Articulation	Shape of the note object (staccato = dot, legato = ribbon, etc.)
Attack	Left edge shape — sharp (hard attack) vs. tapered (soft entry)
Release	Right edge shape — clipped (short) vs. tapered (sustained decay)
Playing Technique	Color hue — normal, pizzicato, col legno, sul ponticello, harmonics
Lane Height	UI option: equal space / instrument range / number of players

3.2 Background Layer — Form and Harmony

Parameter	Visual Encoding
Stable tonal area	Solid color field — one color per key center
Key color vocabulary	Circle of fifths mapped to color wheel; adjacent keys = adjacent hues
Major / minor mode	Warm variant (major) vs. cool variant (minor) of the key color
Modulation passage	Gradient transition between source and target key colors
Section boundaries	Arcs at the top edge of the painting (no vertical bars — preserves continuity)

3.3 Vertical Layout Modes

The system supports two mutually exclusive vertical layout modes, switchable in the UI:

Mode A — Pitch-Based

- Shared vertical pitch space across all instruments
- Higher pitch = higher vertical position, globally

- Instruments playing the same pitch produce overlapping/blending elements
- Encodes sonic reality: what the listener hears
- Best for: revealing doublings, unisons, textural density

Mode B — Score-Based

- Each instrument has its own horizontal lane in vertical order (as in the score)
- Within each lane, elements are positioned high or low according to pitch
- Encodes compositional structure: how the composer organized forces
- Best for: tracing melodic lines across instrument families
- Lane height options: equal / instrument range / number of players

4. Technology Stack

Component	Technology
Score Analysis Pipeline	Python 3.x · music21
Harmonic Analysis	Python 3.x · music21 (key detection, Roman numeral analysis)
Local API Server	Python · FastAPI
Authoring UI Shell	React (JavaScript/TypeScript)
Interactive Canvas / Preview	Konva.js
High-Resolution Export	Python · Cairo (via pycairo)
Interpretation File Format	JSON (Conductor Interpretation Layer — CIL)
Score Import Format	MusicXML (.xml / .mxl)
Export Formats	PDF · PNG · SVG

Deployment: Local web app (Python server + browser UI). Architecture is cloud-deployment-ready — the same codebase can be served publicly by adding cloud storage, authentication, and a deployment target (e.g., Railway + Vercel). Score copyright strategy for public deployment: bring-your-own-file model or curated public domain library.

5. The Conductor Interpretation Layer (CIL)

The Interpretation File is a JSON companion file that attaches to a MusicXML score and encodes conductor-level analytical and interpretive knowledge that MusicXML does not reliably contain: articulation nuance, attack and release character, playing techniques, and phrase intent.

Key design principles:

- The base MusicXML is never modified — the CIL is always a separate file
- The CIL is auto-generated by the Auto-Annotator module as a draft, then refined by the conductor in the Annotation Editor panel
- Multiple conductors can create different CIL files for the same score, producing different visualizations — this is a feature, not a limitation
- The CIL is itself a citable scholarly artifact encoding the conductor's analytical interpretation

Note for future work: The CIL format is a candidate for a standalone publication proposing a community standard for conductor annotation. This paper is scoped for post-tenure development; the CIL concept will be introduced and described in the Seeing the Symphony article as a proposed extension to current music encoding practice.

6. Template System

The rendering engine is modular: the same analyzed data can be piped into different visual layout templates. The template system is designed for extensibility — new templates can be added without modifying the analysis pipeline or data model.

Template A — Piano Roll (Horizontal Panoramic)

- Time flows left to right; pitch occupies the vertical axis
- Horizontal format — well-suited for wide large-format lobby prints
- Familiar to musicians; intuitively readable by non-musicians (high/low is physically natural)
- Lane-based variant (Score Mode) or shared pitch space (Pitch Mode)

Template B — Spiral (Radial)

- Time radiates outward from center; earliest material at center, latest at outer edge
- Naturally encodes musical form — recapitulations and cyclic structures become visually apparent
- More abstract; potentially more striking as a lobby installation piece
- Recommended for works with strong cyclic or arch-form structures

Additional templates may be developed for specific repertoire contexts (e.g., fugue-specific layouts, concerto soloist-vs-orchestra layouts). Template choice is a UI decision made per piece.

6.1 Shape Variant System

Each articulation type is rendered not as a single perfect geometric form but as one of several pre-designed imperfect variants, selected in round-robin rotation. This controlled irregularity is both aesthetically intentional and philosophically coherent: a perfect machine-generated circle would betray the humanistic argument of the installation. The imperfection encodes the fact that music is performed by humans, not computed by machines.

Articulation Type	Shape Variants
Staccato (dot)	4-5 variants: near-circular, slightly flattened, teardrop, irregular blob, soft-edged disc
Legato (ribbon)	4-5 variants: differ in edge softness, slight curvature, pressure simulation along length
Marcato (wedge)	3-4 variants: angle sharpness, slight asymmetry in sides, tip roundness
Tenuto (bar)	3-4 variants: slight taper at ends, minimal curve along spine, edge irregularity
Accent (chevron)	3-4 variants: opening angle, line weight variation, slight skew
Tremolo / sul pont.	4-5 variants: stroke density, angle variation, pressure irregularity

Shape Variant System — Design Principles

- Variants are pre-designed path templates, not runtime noise — the library is curated, not algorithmically generated
- Each variant is a parametric distortion of the canonical shape (slight wobble in outline, simulated brush pressure)
- Variant assignment is deterministic: seeded by note position in the score — same score always renders identically
- The conductor can optionally change the seed value in the UI to explore aesthetic alternatives without re-annotating
- Round-robin cycling ensures no single variant dominates visually across a dense passage
- Variant libraries are extensible — new shapes can be added per template without modifying the selection logic

Implementation note: variants are stored as named Cairo path templates in a shape library module. The round-robin selector is a lightweight pre-render pass that attaches a variant index to each note object in the data model before the renderer draws it.

6.2 Collision Resolution System

When note elements from different instruments occupy the same visual space — most commonly when multiple instruments play at the same pitch — the system applies a

configurable collision resolution strategy. This is a pre-render pass that sits between the Data Model and the Renderer, attaching collision metadata to affected note objects before drawing begins.

Revised pipeline with Collision Resolver:

```
Data Model  ->  Collision Resolver  ->  Render-Ready Objects  ->
Renderer
```

Strategy A — Lane Expansion

- When elements collide, the lane grows vertically to accommodate both without overlap
- Each element retains its full shape and original color — no blending or alternation
- Trade-off: lane heights become dynamic; dense tutti passages cause the lane to expand
- Best for: works where individual instrument identity is the primary visual argument
- Visual effect: the painting breathes and expands at climaxes

Strategy B — Color Alternation

- Overlapping elements alternate their color in a striped or interlaced pattern within the collision zone
- Both instrument colors remain visible and identifiable; no color information is lost
- Trade-off: at high density (large tutti passages) alternation becomes visual noise
- Best used as a per-passage option rather than a global default
- Visual effect: a shimmering, woven texture at points of ensemble convergence

Strategy C — Blending with Protected Core (Recommended Default)

- Each note shape has two concentric zones: a protected core and a blending contour band
- The core retains the original instrument color at full opacity — identity is always preserved
- The contour band participates in color blending with adjacent overlapping elements
- UI slider controls contour percentage: 0% = no blending; 100% = full blend (no protected core)
- Recommended default: 30-40% contour — halo effect where instruments bleed at edges but remain identifiable at center
- Musical analogy: acoustic blend in a live orchestra — instruments merge in the room but remain individually audible

Instrument-Family Awareness (Default Rule Set)

- Blending behavior is family-pair aware: strings with strings use tighter contour than strings with brass
- Default rule set reflects acoustic blend expectations — similar timbres blend more, contrasting timbres blend less

- Conductor can override per family-pair in the UI (not a per-note decision but a per-family-pair preference)
- Family pairs: Strings-Strings / Strings-Woodwinds / Strings-Brass / Woodwinds-Woodwinds / Woodwinds-Brass / Brass-Brass

Implementation note: the Collision Resolver performs spatial overlap detection after the Data Model phase, assigns strategy metadata and contour blend values to affected pairs, and passes render-ready objects to the Renderer. The Renderer applies blend modes (Cairo OVER, MULTIPLY, or custom alpha compositing) per the attached metadata.

7. Development Roadmap

Phase	Deliverable
Phase 0 — Now (Yr 2)	Architecture finalized (this document); project scaffolded in VS Code via Claude Code
Phase 1 — Yr 2–3	Bach chorale prototype: MusicXML parser + Piano Roll renderer + static export
Phase 2 — Yr 3	Harmonic Analyzer + background color layer; conference presentation (NIME or SMT)
Phase 3 — Yr 3–4	Full Authoring UI with live preview, Annotation Editor, template selector
Phase 4 — Yr 4	First lobby installation at Washburn or Overland Park Orchestra concert
Phase 5 — Yr 4–5	Spiral template; user study data; journal article submission
Phase 6 — Yr 5+	Public web deployment; CIL format proposal paper (post-tenure)

8. Scholarly Framing and Tenure Relevance

Seeing the Symphony contributes to the scholarly record in three ways:

- Research Publication: A peer-reviewed article describing the system, the CIL format, and findings from a user study or installation evaluation. Target venues: NIME, Music Perception, SMT.
- Documented Installation: The lobby installation itself is a citable scholarly and artistic output under Washburn's tenure criteria for significant artistic achievement.
- Research Tool: The publicly released authoring tool is a community contribution citable by other researchers, strengthening the publication record through subsequent citations.

Primary research identity reinforced: technology, visual thinking, and computational analysis in service of orchestral pedagogy and accessibility — from the conductor's analytical vantage point.

Appendix: Diagram Files

The following diagram files accompany this document:

- diagram_components.svg — Component architecture diagram (four-layer system with parameter mapping legend)

- diagram_sequence.svg — Data flow sequence diagram (12-step end-to-end workflow)

These diagrams are referenced internally but are also suitable for inclusion in grant applications, conference submissions, and the eventual journal article.